Discrete Wave-Constrained Computation and Classical Complexity: Turing Equivalence for P and NP

Richard J. Reyes

Revised October 27, 2026
Original Release: November 26, 2025

Abstract

Wave Confinement Theory (WCT) motivates a discrete model of computation in which information is stored and processed by local interactions between finitely many metastable states arranged on a lattice. In this paper we isolate the resulting *wave-constrained computation* (WCC) model as a finite-state, finite-range local-update machine and compare its complexity classes to the classical ones.

First, we define a *uniform* discrete WCC machine model with fixed finite alphabet, bounded-radius local updates, polynomial-size constructible lattices, and polynomial-time input encoders and output decoders. We explicitly simulate polynomial-time deterministic Turing machines with constant-factor overhead in the number of global WCC rounds and polynomial overhead in total work; conversely, each such WCC machine has a polynomial-time Turing simulation. Thus $P^{\text{disc}}_{\text{WCC}} = P_{\text{Turing}}$.

We then lift this equivalence from decision procedures to verifier-based nondeterministic computation. We define NP_{WCC} via polynomial-time WCC verifiers with polynomially bounded witnesses, and prove that $NP_{\text{WCC}} = NP_{\text{Turing}}$. Taken together, these results show that the uniform discrete abstraction realizes exactly the classical P and NP classes. Exact symbolic representation of wave trajectories needs

a compatibility condition on quantization and local evolution. Additional physical curvature, energetic and PDE constraints are not covered by the discrete complexity-class equalities; the physical capacity estimates are conditional on their specific dynamical hypotheses.

Kernel verification and model boundary (November 2, 2026). The theorems `PVNP.Discrete.P_WCC_eq_P_TM` and `PVNP.Discrete.NP_WCC_eq_NP_TM` are checked for the specific one-dimensional radius-one finite quiescent model. The manuscript's broader fixed-radius, higher-dimensional and physically realized variants must not be identified with these statements without additional constructions and uniform simulation bounds. In particular, a language-independent polynomial-cost encoder does not make an arbitrary receiver decoder efficient: a fixed noisy sign encoding with n coordinates and squared mass n supports an unrestricted decoder for any predicate. Decoder complexity must be included explicitly in every transfer of an encoding capacity bound into a language-decision claim.

0.1 Resource Model Summary

For convenience, we summarize the resource model used throughout the definition of discrete WCC machines. Let $x \in \Sigma^n$ be an input of length n.

1. **Space (Finite Lattice).** Computation takes place on a finite lattice $\Lambda_n \subset \mathbb{Z}^d$ (with fixed d) satisfying

   $$|\Lambda_n| \leq p(n)$$

   for some polynomial p. The cardinality $|\Lambda_n|$ represents the space cost of the machine.

2. **Precision (Finite State).** The state set S is finite. Each cell holds an element of S, representing at most $\ell = \lceil \log_2 |S| \rceil$ bits of information. No operation increases ℓ; precision remains fixed and finite throughout computation.

3. **Locality (Neighborhood Radius).** A fixed interaction radius $r \in \mathbb{N}$ defines neighborhoods

   $$N_r(i) = \{j \in \Lambda_n : \|i - j\|_\infty \leq r\}.$$

   Each update applies a local rule $U : S^{N_r} \to S$.

4. **Determinism and Time.** Computation proceeds in synchronous, deterministic rounds via U. For a WCC machine W, the time cost on inputs of length n is

$$T_W(n) = \max_{x \in \Sigma^n} \min\{t \geq 0 : \mathrm{dec}(C_t) \in \{0, 1\}\}.$$

   Given C_0 and U, the entire trajectory $(C_t)_{t \geq 0}$ is uniquely determined (deterministic replay).

5. **Total Configuration Space.** Since $|\Lambda_n|$ and $|S|$ are finite, the global configuration space S^{Λ_n} is finite. This precludes hypercomputation via infinite-dimensional state spaces.

The machine descriptions, lattice enumerators, encoders, and decoders are fixed for a language and execute in polynomial time. This uniformity is essential: otherwise an unrestricted input encoding could decide the language before the first WCC update.

Lemma 0.1 (Precision Stability)**.** *Let $W = (\{\Lambda_n\}, S, r, U, \mathrm{enc}, \mathrm{dec})$ be a discrete WCC machine. If $C_t : \Lambda_n \to S$ is representable using $\ell = \lceil \log_2 |S| \rceil$ bits per cell, then so is C_{t+1} obtained by the local update rule U.*

Proof. For each $i \in \Lambda_n$,

$$C_{t+1}(i) = U(C_t|_{N_r(i)}).$$

Since $U : S^{N_r} \to S$ is a total function whose image lies in the same finite state set S, its outputs require no more than $\lceil \log_2 |S| \rceil$ bits. No arithmetic involving unbounded precision is performed. Thus, the precision requirement remains constant, preventing accumulation of infinite information during evolution. □

Proposition 0.2 (Local Implementability of TM Transitions)**.** *Let M be a deterministic single-tape Turing machine with transition function*

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}.$$

Under the two-phase head/symbol encoding of Section 1, each transition of M is realizable by a WCC local update rule with radius $r = 1$.

Proof. In the simulation, each tape cell is encoded as a WCC state $(\gamma, q^{\#}, p) \in S$, where $\gamma \in \Gamma$, $q^{\#} \in Q \cup \{\perp, (q, L), (q, R)\}$ encodes the head state or a directional mark, and $p \in \{0, 1\}$ is a phase bit.

To realize a TM transition, the rule U need only inspect the cell currently holding the head and its immediate neighbors (to which the head may move in the next phase). These cells lie inside the radius–1 neighborhood $\{i - 1, i, i + 1\}$.

Given (q, γ) at the head location, U rewrites the symbol, enters the next state, and places a directional mark for the subsequent phase. Movement of the head by one cell is therefore computed using only local information. Hence, all TM transitions are locally implementable in the WCC model. □

Proposition 0.3 (Deterministic Evolution and Replay)**.** *Let W be a discrete WCC machine. Then the global evolution*

$$C_{t+1}(i) = U(C_t|_{N_r(i)})$$

is deterministic. Consequently, given C_0 and U, the entire trajectory $(C_t)_{t\geq 0}$ is uniquely determined and can be reconstructed (replayed) by any verifier.

Proof. Since U is a function from the finite domain S^{N_r} to the finite set S, each cell update is uniquely determined by the configuration of its neighborhood. Thus C_{t+1} is uniquely determined by C_t.

By induction, the full sequence of configurations is uniquely determined by C_0. A verifier with access to C_0 and U can therefore reproduce the entire evolution without ambiguity, establishing deterministic replay. □

1 Discrete WCC Machines and Turing Equivalence

In this section we isolate the discrete core of wave-constrained computation (WCC) as a finite-state local-update machine model and prove that its polynomial-time class coincides with the classical deterministic polynomial-time class. This makes precise the identification

$$P_{\mathrm{WCC}} = P_{\mathrm{Turing}}$$

used elsewhere in the paper.

We first define the discrete WCC machine under the locality and precision rails, then construct an explicit simulation of any polynomial-time Turing machine by a polynomial-time WCC machine with only constant-factor overhead in time. We then recall the converse simulation (WCC by Turing machines) and obtain full equivalence.

1.1 Discrete WCC machine model

Let Σ be a finite input alphabet and let $n = |x|$ denote input length.

Definition 1.1 (Uniform discrete WCC machine)**.** *A* uniform discrete WCC machine *is a tuple*

$$W = (\{\Lambda_n\}_{n\geq 1}, S, r, U, \mathrm{enc}, \mathrm{dec})$$

with the following data, fixed independently of an individual input.

1. *The spatial dimension $d \geq 1$ and neighborhood radius $r \geq 0$ are constants. The cell set $\Lambda_n \subset \mathbb{Z}^d$ satisfies $|\Lambda_n| \leq p(n)$ and lies in a box with polynomially bounded integer coordinates. A fixed deterministic algorithm constructs and enumerates Λ_n and its fixed-radius neighborhoods in time* $\mathrm{poly}(n)$*.*

2. *S is a fixed, finite alphabet. Boundary cells outside Λ_n read a distinguished fixed background symbol $s_\partial \in S$.*

3. *$U : S^{\{-r,\dots,r\}^d} \to S$ is a single fixed local rule of constant description length. Its synchronous action is*

$$C_{t+1}(i) = U\big((\overline{C}_t(i+v))_{\|v\|_\infty \leq r}\big), \quad i \in \Lambda_n,$$

   where $\overline{C}_t$ extends C_t by the background symbol.

4. *The uniform encoder* enc *constructs $C_0 = \mathrm{enc}(x) \in S^{\Lambda_{|x|}}$ in time* $\mathrm{poly}(|x|)$ *on a standard deterministic Turing machine.*

5. *The uniform decoder $\mathrm{dec}_n : S^{\Lambda_n} \to \{\mathsf{run}, 0, 1\}$ is computed in time* $\mathrm{poly}(n)$ *from the canonical configuration encoding. It returns* run *before halting and* 0 *(reject) or* 1 *(accept) on a halting configuration. Halting outputs are persistent (equivalently the computation is stopped at its first halting configuration).*

The polynomial bounds, fixed rule, and algorithms are part of the machine description, not extra advice indexed by n or an oracle. All complexity classes denoted P_{WCC} and NP_{WCC} below refer to this discrete uniform *model unless a physical superscript is explicitly attached.*

Definition 1.2 (Polynomial-time discrete WCC)**.** *For a uniform discrete WCC machine W, set*

$$T_W(n) = \max_{x\in\Sigma^n} \min\{t \geq 0 : \mathrm{dec}_n(C_t) \in \{0,1\}\}.$$

We require the minimum to exist for every input. The machine decides $L \subseteq \Sigma^*$ in polynomial time if $T_W(n) \leq p(n)$ for some polynomial p and its first halting output equals $\mathbf{1}_L(x)$ for every x. By Definition 1.1, the total cost of constructing, simulating, and decoding each of these runs is polynomial in n whenever the round count is polynomial. We write $P^{\mathrm{disc}}_{\mathrm{WCC}}$, or simply P_{WCC} in the discrete sections, for the resulting class.

The remaining WCC rails (coercivity, entropy pruning, renewability) impose additional structure on how such a discrete machine can arise from an underlying wave-evolution model. In this section we restrict attention to the discrete interface; the physical realization is addressed separately (Remark 1.19).

1.2 Wave–Symbol Correspondence Under Compatibility

The local symbolic update U can faithfully describe a discretized wave evolution only if its quantization commutes with that evolution. Locality and finite precision alone do not imply this compatibility. Here a local wave update is a map $F : Z^{\{-r,\dots,r\}^d} \to Z$ on a finite-precision subset $Z \subset \mathbb{C}$, acting on $\Psi_t : \Lambda_n \to Z$ by

$$\Psi_{t+1}(i) = F\big((\overline{\Psi}_t(i+v))_{\|v\|_\infty \leq r}\big). \tag{1}$$

This is a discrete local map; realizing it by a particular WCT PDE requires independent physical analysis.

Definition 1.3 (Quantization and representative encoding)**.** *Let S be a fixed finite alphabet. Let $Q : Z \to S$ be surjective and $\widetilde{Q} : S \to Z$ select a representative for each symbol, so that*

$$Q \circ \widetilde{Q} = \mathrm{id}_S.$$

Both maps act cellwise on configurations. The symbolic state associated with a field is $C_t = Q \circ \Psi_t$.

Given $F, Q, \widetilde{Q}$, define the candidate symbolic rule

$$U(c) = Q\big(F(\widetilde{Q}(c))\big), \qquad c \in S^{\{-r,\dots,r\}^d}. \tag{2}$$

Theorem 1.4 (Exact wave–symbol correspondence)**.** *Suppose the actual field trajectory obeys* (1) *and that, on every neighborhood configuration z reached by this trajectory, the* quantization compatibility *identity holds:*

$$Q(F(z)) = Q\big(F(\widetilde{Q}(Q(z)))\big). \tag{3}$$

Then $C_t = Q \circ \Psi_t$ satisfies the exact symbolic update

$$C_{t+1}(i) = U\big((\overline{C}_t(i+v))_{\|v\|_\infty \le r}\big). \tag{4}$$

Conversely, for every symbolic local rule U there exists an abstract *finite-precision representative evolution, defined by*

$$F_U(z) = \widetilde{Q}\big(U(Q(z))\big),$$

for which every symbolic trajectory is realized exactly on the invariant code manifold $\widetilde{Q}(S)^{\Lambda_n}$. This converse is a statement about a constructed finite-state local evolution, not about solutions of an independently fixed continuous WCT PDE.

Proof. For an actual trajectory and each site i, write z for the corresponding input neighborhood. By compatibility,

$$Q(\Psi_{t+1}(i)) = Q(F(z)) = Q(F(\widetilde{Q}(Q(z)))) = U(Q(z)),$$

which is (4). For the converse, initialize $\Psi_0 = \widetilde{Q}(C_0)$ and use F_U. If $\Psi_t = \widetilde{Q}(C_t)$, then

$$\Psi_{t+1} = \widetilde{Q}\big(U(Q(\Psi_t))\big) = \widetilde{Q}(U(C_t)) = \widetilde{Q}(C_{t+1}).$$

Induction proves exact representative dynamics and invariance of the code manifold. No continuous curvature or energy bound is inferred. □

Remark 1.5 (Compatibility is necessary)**.** *Without* (3)*, quantization need not commute with a local update. For example, let* $Z = \{-1, \frac{1}{2}, 1\}$ *and* $S = \{0, 1\}$*, with* $Q(z) = \mathbf{1}_{\{z>0\}}$*,* $\widetilde{Q}(0) = -1$*,* $\widetilde{Q}(1) = 1$*. Define the one-cell map* $F(-1) = -1$*,* $F(\frac{1}{2}) = \frac{1}{2}$*, and* $F(1) = -1$*. For* $\Psi_0 = \frac{1}{2}$*, one has* $Q(F(\Psi_0)) = 1$*, whereas* $U(Q(\Psi_0)) = Q(F(1)) = 0$*. Thus the unqualified wave–symbol isomorphism is false, even for single-site local maps. Exact correspondence requires the stated compatibility or an invariant representative code manifold.*

1.3 Gandy Correspondence and Physical Cellular Automata

Gandy's physical Church–Turing thesis characterizes a broad class of "reasonable" physical computation models by four structural axioms: bounded information density, spatial homogeneity, bounded propagation speed, and local causality. In this subsection we show that discrete WCC machines satisfy these axioms and therefore instantiate a physical cellular automaton in Gandy's sense.

Definition 1.6 (Gandy-style physical axioms)**.** *A discrete dynamical system on configurations* $C : \Lambda \to S$ *satisfies Gandy's physical constraints if:*

1. ***Bounded information density:*** *S is finite, so each site carries at most* $\log_2 |S|$ *bits of information.*
2. ***Spatial homogeneity:*** *There exists a fixed local rule* $U : S^{N_r} \to S$ *independent of the site* $i \in \Lambda$ *and of the configuration size.*
3. ***Bounded propagation:*** *There is a fixed radius* $r \in \mathbb{N}$ *such that the update at site* i *depends only on the values in the neighbourhood* $N_r(i)$*.*
4. ***Local causality:*** *The global update* $C \mapsto C'$ *is defined pointwise by*

$$C'(i) = U\big(C|_{N_r(i)}\big), \qquad i \in \Lambda,$$

with no dependence on nonlocal aggregates.

Proposition 1.7 (WCC satisfies Gandy axioms)**.** *Let*

$$W = (\{\Lambda_n\}_{n\geq 1}, S, r, U, \mathrm{enc}, \mathrm{dec})$$

be a discrete WCC machine as in Definition 1.1. Then for each n the induced evolution on configurations $C_t : \Lambda_n \to S$ satisfies all four Gandy-style physical axioms.

Proof. Fix $n \geq 1$.

(1) *Bounded information density.* By definition of a WCC machine, the state set S is finite. Thus each cell carries at most $\log_2 |S|$ bits of information.

(2) *Spatial homogeneity.* The local update rule $U : S^{N_r} \to S$ is fixed once and for all in the specification of W, and does not depend on $i \in \Lambda_n$ or on n.

(3) *Bounded propagation.* The neighbourhood shape $N_r(i)$ is defined with a fixed radius $r \in \mathbb{N}$. Therefore the update at site i cannot depend on configuration values outside $N_r(i)$, and information cannot propagate faster than r cells per global step.

(4) *Local causality.* By definition of the WCC evolution,

$$C_{t+1}(i) = U\big(C_t|_{N_r(i)}\big),$$

so the future value at site i depends only on the local neighbourhood $N_r(i)$ at time t and not on any global property of C_t.

Thus all four axioms hold for the evolution on Λ_n, for each n. □

Corollary 1.8 (Gandy–WCC correspondence)**.** *Every discrete WCC machine induces, for each input length n, a finite-state cellular-automaton-style evolution satisfying the listed structural locality axioms. This is a statement about the abstract machine, not a proof of realizability by any fixed physical PDE.*

Proposition 1.7 and Corollary 1.8 show that uniform WCC is a discrete abstraction compatible with the structural locality axioms. Physical feasibility and energy accounting are independent questions.

1.4 Deterministic Turing machines

We recall the standard deterministic Turing machine model and its polynomial-time class.

Definition 1.9 (Deterministic Turing machine)**.** *A deterministic Turing machine is a tuple*

$$M = (Q, \Gamma, \Sigma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

where:

- Q *is a finite set of states.*
- Γ *is a finite tape alphabet containing a distinguished blank symbol* $\sqcup$*.*
- $\Sigma \subseteq \Gamma \setminus \{\sqcup\}$ *is the input alphabet.*
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ *is the transition function.*
- $q_0 \in Q$ *is the start state.*
- $q_{\text{acc}}, q_{\text{rej}} \in Q$ *are halting accept/reject states.*

A configuration *of* M *is a triple* (γ, h, q) *consisting of a bi-infinite tape* $\gamma : \mathbb{Z} \to \Gamma$ *with all but finitely many entries equal to* $\sqcup$*, a head position* $h \in \mathbb{Z}$*, and a current state* $q \in Q$*. One transition step applies* $\delta(q, \gamma(h))$ *and updates* (γ, h, q) *accordingly.*

Definition 1.10 (Polynomial-time Turing computation)**.** *Let* $T_M(n)$ *denote the worst-case number of transition steps that* M *uses on inputs of length* n*. A language* $L \subseteq \Sigma^*$ *is in* P_{Turing} *if there exists a machine* M *and constants* $c, k \geq 1$ *such that*

$$T_M(n) \leq cn^k$$

for all n*, and* M *accepts* x *if and only if* $x \in L$*.*

1.5 Simulation of polynomial-time Turing machines by WCC

We now construct, for each polynomial-time Turing machine M, a discrete WCC machine W_M that simulates M with only constant-factor overhead in time. The construction proceeds via a one-dimensional radius-1 local update rule that encodes the tape and head in per-cell symbols and uses a two-phase schedule to implement the Turing transition function.

1.5.1 Configuration encoding

Fix a Turing machine M as in Definition 1.9. We first define the local alphabet used by the WCC machine.

Definition 1.11 (Local tape state and phase)**.** *Let*

$$S_{\text{tape}} := \Gamma \times (Q \cup \{\bot\})$$

where the symbol $\bot$ denotes "no head present". A local tape state $(\gamma, q) \in S_{\text{tape}}$ thus records the tape symbol $\gamma \in \Gamma$ and, if $q \in Q$, the presence of the head with state q at the corresponding cell.

We extend the state space with a phase bit and a directional mark. Let

$$Q' := Q \cup \{(q, L) : q \in Q\} \cup \{(q, R) : q \in Q\}$$

and define

$$S := \Gamma \times (Q' \cup \{\bot\}) \times \{0, 1\}.$$

An element $s = (\gamma, q^{\#}, p) \in S$ consists of:

- *a symbol $\gamma \in \Gamma$,*
- *either $q^{\#} = \bot$ (no head), or $q^{\#} \in Q$ (head present with state $q^{\#}$), or $q^{\#} = (q, d)$ with $q \in Q$ and $d \in \{L, R\}$ indicating that in the next phase the head should move one cell in direction d,*
- *a phase bit $p \in \{0, 1\}$. Phase $p = 0$ corresponds to "before Turing transition" and phase $p = 1$ to "after symbol rewrite, before head movement".*

Given a Turing configuration (γ, h, q) at time t, we encode it as a configuration $C_t : \mathbb{Z} \to S$ by

$$C_t(i) := \begin{cases} (\gamma(i), q, 0), & \text{if } i = h, \\ (\gamma(i), \bot, 0), & \text{if } i \neq h. \end{cases}$$

In the WCC machine we restrict to a finite window

$$\Lambda_n := \{-L(n), \ldots, L(n)\} \subset \mathbb{Z}$$

with $L(n)$ chosen as a polynomial upper bound on the space usage of M on inputs of length n, for example

$$L(n) := T_M(n) + n.$$

1.5.2 Local update rule

We now define the local rule $U : S^{\{-1,0,1\}} \to S$, which acts with radius $r = 1$ in dimension $d = 1$. For each site $i \in \Lambda_n$, the neighbourhood is

$$N_1(i) = \{i - 1, i, i + 1\}$$

(with the understanding that we only care about neighbourhoods fully contained in Λ_n; since the head never reaches the boundary within $T_M(n)$ steps, the behaviour of U at the outermost sites can be defined arbitrarily).

The action of U depends on the phase bit of the centre cell.

Phase 0: symbol rewrite and direction tagging. In phase $p = 0$, at most one cell carries a head state in Q, all other cells have $q^{\#} = \perp$. For a cell at site i with local state $s_i = (\gamma_i, q_i^{\#}, 0)$, we define the phase-0 component U_{phase0} as:

- If $q_i^{\#} \in Q$ (the head is present at i with state $q_i^{\#}$), we consult the Turing transition function

  $$\delta(q_i^{\#}, \gamma_i) = (q', \gamma', d), \qquad d \in \{L, R\},$$

  and set

  $$U_{\text{phase0}}(s_i) := (\gamma', (q', d), 1),$$

  that is, we overwrite the symbol with γ', store a directional mark (q', d), and flip the phase bit to 1.

- If $q_i^{\#} = \perp$ (no head), then the cell does not change its symbol or head component and only flips the phase:

  $$U_{\text{phase0}}(s_i) := (\gamma_i, \perp, 1).$$

This part of the rule is radius-0; it will be incorporated into the full radius-1 rule below.

Phase 1: head movement. In phase $p = 1$, exactly one cell in a valid configuration carries a directional mark $(q', d) \in Q'$ and all others carry either $\perp$ or an unmarked $q \in Q$. The head must move one cell in direction d, and the directional mark must be cleared.

The update for phase 1 uses the radius-1 neighbourhood

$$(C(i-1), C(i), C(i+1)) = (s_{i-1}, s_i, s_{i+1}) \in S^3.$$

We write $s_j = (\gamma_j, q_j^{\#}, 1)$ with $q_j^{\#} \in Q' \cup \{\perp\}$ and define $U_{\text{phase1}} : S^3 \to S$ by case distinction:

- If the *centre* cell carries a right-moving mark, i.e. $q_i^{\#} = (q', R)$, then the head is leaving site i:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, \perp, 0).$$

- If the *left neighbour* carries a right-moving mark and the centre cell is headless, i.e. $q_{i-1}^{\#} = (q', R)$ and $q_i^{\#} = \perp$, then the head arrives from the left:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q', 0).$$

- If the centre cell carries a left-moving mark, $q_i^{\#} = (q', L)$, then the head is leaving site i to the left:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, \perp, 0).$$

- If the *right neighbour* carries a left-moving mark and the centre cell is headless, $q_{i+1}^{\#} = (q', L)$ and $q_i^{\#} = \perp$, then the head arrives from the right:

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q', 0).$$

- In all remaining cases (cells that neither carry nor receive a head), the cell preserves its tape symbol and any non-directional head component and simply flips the phase bit $1 \mapsto 0$. Under the invariant stated below, this means

$$U_{\text{phase1}}(s_{i-1}, s_i, s_{i+1}) := (\gamma_i, q_i^*, 0)$$

  where q_i^* is either $\perp$ or $q \in Q$ (never a directional mark).

The full local rule U is obtained by combining the two phases into a single radius-1 rule on $S^{\{-1,0,1\}}$, with the phase bit p in the centre cell determining which branch, U_{phase0} or U_{phase1}, applies at that cell.

1.5.3 Encoding, decoding, and correctness

We now define the WCC machine W_M associated to M.

Definition 1.12 (Associated WCC machine W_M)**.** *Let M be a deterministic Turing machine as above. Define W_M to be the discrete WCC machine with:*

- *dimension $d = 1$,*
- *index sets $\Lambda_n = \{-L(n), \dots, L(n)\}$ with $L(n)$ a fixed polynomial upper bound on M's tape usage on inputs of length n,*
- *state set S as in Definition 1.11 and radius $r = 1$, with neighbourhoods $N_1(i) = \{i-1, i, i+1\}$,*
- *local update rule $U : S^{\{-1,0,1\}} \to S$ as defined above,*
- *encoding $\mathrm{enc}_M : \Sigma^* \to S^{\Lambda_n}$ given by*

$$\mathrm{enc}_M(x)_i = \begin{cases} (x_1, q_0, 0), & i = 0, \\ (x_{i+1}, \perp, 0), & 1 \le i \le n-1, \\ (\sqcup, \perp, 0), & \text{otherwise}, \end{cases}$$

  i.e. the input word $x = x_1 \dots x_n$ is written on cells $0, \dots, n-1$, the head starts at $i = 0$ in state q_0, and the rest of the tape is blank,

- *decoding predicate $\mathrm{dec}_M : S^{\Lambda_n} \to \{\mathsf{run}, 0, 1\}$ defined by*

$$\mathrm{dec}_M(C) = \begin{cases} 1, & \text{if the simulated head is in } q_{\mathrm{acc}}, \\ 0, & \text{if the simulated head is in } q_{\mathrm{rej}}, \\ \mathsf{run}, & \text{otherwise.} \end{cases}$$

We first state an invariant that characterizes valid configurations in the simulation.

Lemma 1.13 (Simulation invariant)**.** *For the WCC machine W_M on input $x \in \Sigma^n$, the following holds for all $t \ge 0$:*

- *At every* even *time $2t$, all cells have phase bit $p = 0$ and there is a unique head cell $h_t \in \Lambda_n$ such that $C_{2t}(h_t) = (\gamma_t(h_t), q_t, 0)$ with $q_t \in Q$, while all other cells have $q^{\#} = \perp$.*

- *At every* odd *time* $2t+1$*, all cells have phase bit* $p=1$ *and there is a unique directional mark at some site* $j_t \in \Lambda_n$ *with* $C_{2t+1}(j_t) = (\widetilde{\gamma}(j_t), (q', d), 1)$ *for some* $q' \in Q$ *and* $d \in \{L, R\}$*, while all other cells have* $q^{\#} \in Q \cup \{\perp\}$ *but never a directional mark.*

Proof. The base case follows from the definition of enc_M at $t = 0$. The step from $2t$ to $2t+1$ uses U_{phase0}, which can only introduce a single directional mark at the unique head cell and flips all phases to 1. The step from $2t+1$ to $2t+2$ uses U_{phase1}, which ensures that any directional mark at the centre cell is cleared and exactly one neighbour installs the corresponding non-directional head state; all other cells remain without directional marks and all phases flip back to 0. The uniqueness of the marked site follows from the fact that there is exactly one directional mark at odd times and the local rule never creates additional marks; a straightforward case analysis on the neighbourhoods shows that no two cells can simultaneously install a head state from the same marked neighbour. This preserves the invariant. □

We now establish configuration correspondence and runtime.

Lemma 1.14 (One Turing step equals two WCC steps)**.** *Let* (γ_t, h_t, q_t) *be the Turing configuration of* M *after* t *steps on input* x*, and let* C_{2t} *be the configuration of* W_M *after* $2t$ *update steps with initial condition* $C_0 = \mathrm{enc}_M(x)$*. Then, for all* $t \geq 0$*, the configuration* C_{2t} *encodes* (γ_t, h_t, q_t) *in the sense that:*

- *For all* $i \in \Lambda_n$*, the tape symbol in* $C_{2t}(i)$ *equals* $\gamma_t(i)$*.*
- *There exists a unique* $i = h_t$ *such that the head component of* $C_{2t}(i)$ *equals* q_t *and all other cells carry* $\perp$ *in the head component.*
- *All cells have phase bit* $p = 0$*.*

Moreover, applying two further WCC update steps transforms C_{2t} *into* $C_{2(t+1)}$ *which encodes* $(\gamma_{t+1}, h_{t+1}, q_{t+1})$*.*

Proof. The claim follows by induction on t. For $t = 0$ the statement holds by construction of enc_M and the invariant of Lemma 1.13. Suppose the claim holds for some $t \geq 0$. In configuration C_{2t} there is exactly one cell h_t with head state q_t at phase $p = 0$, and the tape symbols agree with $\gamma_t(i)$ for all i.

The phase-0 part of the local rule applies $\delta(q_t, \gamma_t(h_t)) = (q', \gamma', d)$ at cell h_t, overwriting the tape symbol with γ' and marking the head component as (q', d) while flipping the phase to 1. All other cells preserve their tape

symbols and have no head, and flip their phase to 1. After this first update, we obtain C_{2t+1}, in which the tape symbols coincide with those of γ_t except at h_t, where they match the updated symbol γ', and the unique directional head mark (q', d) is at cell h_t, in accordance with Lemma 1.13.

In the next update step (phase 1), the radius-1 rule U_{phase1} moves the directional head mark one cell in direction d to obtain a unique head state q' at either $h_t + 1$ (if $d = R$) or $h_t - 1$ (if $d = L$), clears the directional mark, and restores all phase bits to 0, producing $C_{2(t+1)}$. The tape symbols are unchanged in this step. By definition of the Turing transition, the new Turing configuration is $(\gamma_{t+1}, h_{t+1}, q_{t+1})$ with γ_{t+1} and (h_{t+1}, q_{t+1}) matching the updated symbol and head location/state. Thus $C_{2(t+1)}$ encodes $(\gamma_{t+1}, h_{t+1}, q_{t+1})$ as claimed. This completes the induction. □

Lemma 1.15 (Runtime and space overhead)**.** *Let $T_M(n)$ be the worst-case runtime of M on inputs of length n. Then the associated WCC machine W_M satisfies:*

- *For all $x \in \Sigma^n$, W_M reaches an accepting configuration if and only if M enters q_{acc} on x.*
- *The number of WCC update steps until halting satisfies*

$$T_{W_M}(n) = 2T_M(n).$$

- *The number of cells satisfies $|\Lambda_n| \leq 2L(n) + 1 \leq \tilde{p}(n)$ for some polynomial $\tilde{p}$.*

Proof. The first two claims follow directly from Lemma 1.14 and the definition of dec_M: after $T_M(n)$ Turing steps, M halts in either q_{acc} or q_{rej}, and the corresponding encoded configuration appears at WCC time $2T_M(n)$ with phase $p = 0$; the decoding predicate tests for the presence of q_{acc} and thus detects acceptance exactly when M accepts.

For the space bound, by hypothesis M uses at most $L(n)$ nonblank tape cells on inputs of length n, so choosing $\Lambda_n = \{-L(n), \ldots, L(n)\}$ suffices. Since $L(n)$ is bounded by a polynomial in n, we have $|\Lambda_n| = 2L(n)+1 \leq \tilde{p}(n)$ for a polynomial $\tilde{p}$. Each update step applies the local rule U independently at each cell, so a single global update can be carried out with work $O(|\Lambda_n|) = \text{poly}(n)$, which respects the polynomial-time cost per pass assumed in Definition 1.2. □

We can now state the main equivalence theorem for the simulation direction TM $\Rightarrow$ WCC.

Theorem 1.16 (Simulation of polynomial-time Turing machines by WCC)**.** *Let $L \subseteq \Sigma^*$ be a language decided by a deterministic Turing machine M in time $T_M(n) \leq cn^k$ for some constants $c, k \geq 1$. Then the associated WCC machine W_M defined in Definition 1.12 decides L in time*

$$T_{W_M}(n) = 2T_M(n) \leq 2cn^k$$

on inputs of length n. In particular,

$$P_{\text{Turing}} \subseteq P_{\text{WCC}}.$$

Proof. By Lemma 1.15, W_M accepts exactly the inputs accepted by M and does so in at most $2T_M(n)$ global update steps on inputs of length n. The bound $T_M(n) \leq cn^k$ implies $T_{W_M}(n) \leq 2cn^k$, so W_M is a polynomial-time WCC machine in the sense of Definition 1.2. Therefore $L \in P_{\text{WCC}}$. Since L was arbitrary in P_{Turing}, we obtain $P_{\text{Turing}} \subseteq P_{\text{WCC}}$. □

1.6 Simulation of WCC by Turing machines

We now recall the converse direction: any discrete WCC machine can be simulated by a deterministic Turing machine with polynomial overhead. This is essentially a standard "Turing simulation of a cellular automaton" argument.

Theorem 1.17 (Simulation of WCC by Turing machines)**.** *Let W be a discrete WCC machine that decides a language L in polynomial time as in Definition 1.2, with runtime bound $T_W(n) \leq n^k$ and $|\Lambda_n| \leq p(n)$ for some polynomials k, p. Then there exists a deterministic Turing machine M_W that decides L in time polynomially bounded in n. In particular,*

$$P_{\text{WCC}} \subseteq P_{\text{Turing}}.$$

Proof. We sketch the standard simulation; details can be made fully explicit if desired.

Fix n and the corresponding index set Λ_n. A configuration $C_t : \Lambda_n \to S$ of W can be encoded on the tape of a Turing machine as a finite word over $\Gamma_{\text{enc}} := S \cup \{\#\}$ by listing the cell states in some canonical order $i_1, \ldots, i_{|\Lambda_n|}$

(e.g. lexicographic order on Λ_n) separated by a delimiter #. This yields an encoding code(C_t) of length $O(|\Lambda_n|)$.

To simulate one global WCC update $C_t \mapsto C_{t+1}$, the Turing machine M_W maintains the current configuration encoding on its work tape and proceeds in two phases:

1. It copies the current configuration to an auxiliary work area. This takes time $O(|\Lambda_n|)$.

2. It enumerates each cell, uses the uniform lattice/neighborhood algorithms to retrieve its at most $(2r+1)^d$ local symbols, computes the fixed rule U, and writes the next state. The local lookup and update have polynomial cost in n; a simple tape implementation may take $O(|\Lambda_n|^2 \operatorname{poly}(n))$ time per round. No constant-time random access is assumed.

By assumption $|\Lambda_n| \leq p(n)$ and one global update can be effected with cost polynomial in $|\Lambda_n|$, i.e. poly$(p(n))$. Repeating this procedure for $T_W(n) \leq n^k$ steps yields an overall Turing runtime

$$T_{M_W}(n) \leq T_W(n) \cdot \operatorname{poly}(|\Lambda_n|) \leq n^k \cdot \operatorname{poly}(p(n)) = \operatorname{poly}(n).$$

The Turing machine M_W accepts exactly when the simulated configuration satisfies $\operatorname{dec}(C_t) = 1$, which can be checked during or after each simulated global update by scanning the encoding for an accepting pattern. Thus M_W decides the same language L in polynomial time, and $L \in P_{\text{Turing}}$. Since L was arbitrary in P_{WCC}, it follows that $P_{\text{WCC}} \subseteq P_{\text{Turing}}$. □

1.7 Equivalence of P_{WCC} and P_{Turing}

Combining Theorem 1.16 and Theorem 1.17 yields the desired identification.

Corollary 1.18 (Polynomial-time equivalence)**.** *Under the discrete WCC machine model of Definition 1.1 and the standard deterministic Turing machine model of Definition 1.9, the polynomial-time classes coincide:*

$$P_{\text{WCC}} = P_{\text{Turing}}.$$

Remark 1.19 (Physical WCT realization is a separate obligation)**.** *The equality $P^{\text{disc}}_{\text{WCC}} = P$ concerns only the uniform finite-state abstraction of Definition 1.1. It does* not *imply $P^{\text{phys}}_{\text{WCC}} = P$. To deduce that equality for a*

curvature-constrained WCT device family, one must additionally construct, uniformly in input length and for every required local rule, a stable encoding $s \mapsto \psi_s$ and a physical evolution satisfying:

1. *the exact code-state transition compatibility of Theorem 1.4;*
2. *polynomially bounded preparation, run time, volume, energy/curvature expenditure, precision and readout costs;*
3. *correct uniform simulation of all polynomial-time discrete WCC computations, including their verifier machines.*

These are additional model-specific hypotheses. They are neither consequences of locality and finite precision nor established by the Turing simulations proved in this section. The reverse inclusion $P^{\mathrm{phys}}_{\mathrm{WCC}} \subseteq P$ would likewise require a precise representation and polynomial-time classical simulation of all admissible physical operations.

2 WCC Verifiers and NP Equivalence

In this section we lift the polynomial-time equivalence of Corollary 1.18 from decision procedures to verifier-based nondeterministic computation. We define an NP-type class NP_{WCC} using polynomial-time WCC verifiers and show that, under the same discrete model assumptions, it coincides with the classical nondeterministic polynomial-time class NP_{Turing}.

2.1 Classical NP via Turing verifiers

We first recall the standard verifier-based definition of NP.

Let Σ be a finite input alphabet and let Γ_{wit} be a finite witness alphabet. For $x \in \Sigma^n$ and $y \in \Gamma_{\mathrm{wit}}^m$ we write (x, y) for the pair instance, and $N := n+m$ for the total input length.

Definition 2.1 (Polynomial-time Turing verifier)**.** *A* polynomial-time Turing verifier *for a language $L \subseteq \Sigma^*$ is a deterministic Turing machine*

$$V_{\mathrm{T}} = (Q, \Gamma, \Sigma \cup \Gamma_{\mathrm{wit}}, \delta, q_0, q_{\mathrm{acc}}, q_{\mathrm{rej}})$$

such that:

- *V_{T} takes as input a pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\mathrm{wit}}^m$, encoded in some fixed, efficiently decodable format on its tape.*
- *There exist polynomials p, q such that for all $x \in \Sigma^n$ and all y,*

$$T_{V_{\mathrm{T}}}(|x| + |y|) \leq p(|x| + |y|)$$

and

$$x \in L \iff \exists y \in \Gamma_{\mathrm{wit}}^m,\ m \leq q(|x|) \text{ such that } V_{\mathrm{T}}(x, y) \text{ accepts.}$$

Definition 2.2 (Classical nondeterministic polynomial time)**.** *A language $L \subseteq \Sigma^*$ is in NP_{Turing} if there exists a polynomial-time Turing verifier V_{T} for L in the sense of Definition 2.1. The class NP_{Turing} is the set of all such languages.*

This definition is equivalent to the usual one via nondeterministic Turing machines, but we will not need that equivalence here.

2.2 WCC verifiers and NP_{WCC}

We now define polynomial-time WCC verifiers and the associated NP-type class.

Definition 2.3 (Polynomial-time WCC verifier)**.** *Let Σ be a finite input alphabet and Γ_{wit} a finite witness alphabet. A* polynomial-time WCC verifier *for a language $L \subseteq \Sigma^*$ is a discrete WCC machine*

$$V_{\mathrm{W}} = (\{\Lambda_N\}_{N \geq 1}, S, r, U, \mathrm{enc}_{\mathrm{W}}, \mathrm{dec}_{\mathrm{W}})$$

in the sense of Definition 1.1, together with polynomials p, q such that:

- *For each instance $x \in \Sigma^n$ and candidate witness $y \in \Gamma_{\mathrm{wit}}^m$ with $m \leq q(n)$, the pair (x, y) is encoded as an initial configuration*

$$C_0 = \mathrm{enc}_{\mathrm{W}}(x, y) \in S^{\Lambda_N}, \qquad N := n + m,$$

for some index set Λ_N with $|\Lambda_N| \leq p(N)$.

- *The WCC runtime $T_{V_{\mathrm{W}}}(N)$, defined as the worst-case number of global update steps until $\mathrm{dec}_{\mathrm{W}}(C_t)$ first returns 0 or 1 on inputs of total length N, satisfies*

$$T_{V_{\mathrm{W}}}(N) \le p(N)$$

  for all N.

- *For all $x \in \Sigma^n$,*

$$x \in L \iff \exists y \in \Gamma_{\mathrm{wit}}^m,\ m \le q(n) \text{ such that } V_{\mathrm{W}}(x,y) \text{ accepts,}$$

  where acceptance means $\mathrm{dec}_{\mathrm{W}}(C_t) = 1$ for some $t \le T_{V_{\mathrm{W}}}(|x| + |y|)$.

Definition 2.4 (WCC nondeterministic polynomial time)**.** *A language $L \subseteq \Sigma^*$ is in NP_{WCC} if there exists a polynomial-time WCC verifier V_{W} for L in the sense of Definition 2.3. The class NP_{WCC} is the set of all such languages.*

By construction, NP_{WCC} is defined using exactly the same witness-length and runtime scaling as NP_{Turing}, but with WCC machines in place of Turing machines.

2.3 Equivalence of NP_{WCC} and NP_{Turing}

We now prove that, under the discrete WCC model, the verifier-based NP class coincides with the classical one.

Theorem 2.5 (NP equivalence)**.** *Under the discrete WCC machine model of Definition 1.1 and the Turing verifier model of Definition 2.1, the nondeterministic polynomial-time classes coincide:*

$$NP_{\mathrm{WCC}} = NP_{\mathrm{Turing}}.$$

Proof. We prove both inclusions.

Inclusion $NP_{\mathrm{WCC}} \subseteq NP_{\mathrm{Turing}}$. Suppose $L \in NP_{\mathrm{WCC}}$. Then there exists a polynomial-time WCC verifier V_{W} for L with polynomials p, q as in Definition 2.3. For each pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\mathrm{wit}}^m$, $m \le q(n)$, the machine V_{W} runs in at most $p(N)$ global update steps, where $N := n + m$, on a lattice Λ_N with $|\Lambda_N| \le p(N)$.

By Theorem 1.17, there exists a deterministic Turing machine $M_{V_{\mathrm{W}}}$ that simulates V_{W} on inputs of total length N in time bounded by

$$T_{M_{V_{\mathrm{W}}}}(N) \leq p(N) \cdot \mathrm{poly}(|\Lambda_N|) \leq p(N) \cdot \mathrm{poly}(p(N)) = \mathrm{poly}(N).$$

Define a Turing verifier V_{T} that, on input (x, y), first constructs the encoding $\mathrm{enc}_{\mathrm{W}}(x, y)$ on its tape and then simulates $M_{V_{\mathrm{W}}}$ for $p(N) \cdot \mathrm{poly}(p(N))$ steps, accepting if and only if $V_{\mathrm{W}}(x, y)$ accepts. This adds at most an additional polynomial overhead in N for the encoding step, so V_{T} runs in time polynomial in N.

By construction, for either verifier V in $\{V_{\mathrm{W}}, V_{\mathrm{T}}\}$ the same predicate is decided:

$$x \in L \quad \Longleftrightarrow \quad \exists y \, \big(|y| \leq q(|x|) \ \wedge \ V(x, y) = 1\big).$$

Thus V_{T} is a polynomial-time Turing verifier for L in the sense of Definition 2.1, and hence $L \in NP_{\mathrm{Turing}}$. Since L was arbitrary in NP_{WCC}, we obtain $NP_{\mathrm{WCC}} \subseteq NP_{\mathrm{Turing}}$.

Inclusion $NP_{\mathrm{Turing}} \subseteq NP_{\mathrm{WCC}}$. Conversely, suppose $L \in NP_{\mathrm{Turing}}$. Then there exists a polynomial-time Turing verifier V_{T} for L with polynomials p, q as in Definition 2.1. For each pair (x, y) with $x \in \Sigma^n$, $y \in \Gamma_{\mathrm{wit}}^m$, $m \leq q(n)$, the machine V_{T} halts in at most $p(N)$ steps with $N := n + m$.

Apply the construction of Theorem 1.16 to V_{T}, treating (x, y) as a single input string of length N. This yields a discrete WCC machine $W_{V_{\mathrm{T}}}$ that simulates V_{T} with at most a constant-factor overhead in time, i.e.

$$T_{W_{V_{\mathrm{T}}}}(N) \leq c\, T_{V_{\mathrm{T}}}(N) \leq c\, p(N)$$

for some constant $c \geq 1$, on index sets Λ_N with $|\Lambda_N| \leq \tilde{p}(N)$ for some polynomial $\tilde{p}$. We define the encoding map $\mathrm{enc}_{\mathrm{W}}$ for $W_{V_{\mathrm{T}}}$ by viewing (x, y) simply as a word over the combined alphabet $\Sigma \cup \Gamma_{\mathrm{wit}}$ and placing it on the initial tape segment as in Definition 1.12. The decoding predicate $\mathrm{dec}_{\mathrm{W}}$ tests for the presence of the accepting state q_{acc} in the head component, exactly as in Definition 1.12.

By Lemma 1.15 and Theorem 1.16, we then have

$$(x, y) \text{ accepted by } V_{\mathrm{T}} \iff (x, y) \text{ accepted by } W_{V_{\mathrm{T}}}$$

for all pairs (x, y). Thus, with V equal to either V_{T} or $W_{V_{\mathrm{T}}}$,

$$x \in L \quad \Longleftrightarrow \quad \exists y \, \big(|y| \leq q(|x|) \ \wedge \ V(x, y) = 1\big).$$

Thus $W_{V_{\mathrm{T}}}$ is a polynomial-time WCC verifier for L with witness bound $q(\cdot)$ and runtime bounded by a polynomial in N, and hence $L \in NP_{\mathrm{WCC}}$.

Since L was arbitrary in NP_{Turing}, we obtain $NP_{\mathrm{Turing}} \subseteq NP_{\mathrm{WCC}}$.

Conclusion. We have shown $NP_{\mathrm{WCC}} \subseteq NP_{\mathrm{Turing}}$ and $NP_{\mathrm{Turing}} \subseteq NP_{\mathrm{WCC}}$, hence

$$NP_{\mathrm{WCC}} = NP_{\mathrm{Turing}}.$$

This completes the proof. □

3 Curvature-Bounded Channel Capacity in WCT

In this section we make precise the intuition that a wave-confinement device with finite volume, energy budget, and runtime cannot sustain arbitrarily many independent information "lanes" or arbitrarily fast switching between them. Under the rest-energy curvature identity and the dimensional lock on timescales, we obtain a polynomial upper bound on the spatiotemporal information-processing capacity of a WCT cavity.

3.1 Geometric and energetic setup

Let Γ be a smooth closed characteristic loop of a confined field configuration with arclength parameter $s \in [0, L)$, total length

$$L = \oint_{\Gamma} ds,$$

and Frenet–Serret curvature $\kappa(s)$ and torsion $\tau(s)$. We define the local curvature-rate scalar

$$\sigma(s)^2 := \kappa(s)^2 + \tau(s)^2. \tag{5}$$

Let $w(s) > 0$ be a weight function induced, for example, by an energy density or probability density derived from $|\psi|^2$ along Γ. We write the corresponding weighted average of a quantity f as

$$\langle f \rangle_w := \frac{\oint_{\Gamma} w(s)\, f(s)\, ds}{\oint_{\Gamma} w(s)\, ds}. \tag{6}$$

Definition 3.1 (Rest energy / mass from curvature)**.** *We say that a confined field configuration obeys the* rest energy curvature identity *if its rest energy and inertial mass are given by*

$$E_{\text{rest}} = \hbar c \, \langle \sigma \rangle_w, \qquad m = \frac{\hbar}{c} \, \langle \sigma \rangle_w, \tag{7}$$

for some positive weight w along closed characteristic loops of the configuration.

In a bulk description, we consider a bounded cavity $\Omega \subset \mathbb{R}^3$ with volume $V := |\Omega|$ confining a complex scalar field

$$\psi : \Omega \times \mathbb{R} \to \mathbb{C},$$

which we assume satisfies a WCT-type curvature-locking PDE. We denote by $E_{\text{rest}}(\psi)$ the total rest energy of ψ in Ω and impose a resource bound

$$E_{\text{rest}}(\psi) \leq E_{\max}. \tag{8}$$

3.2 Curvature, bandlimit, and spatial capacity

We first connect the curvature/energy budget to an effective bandlimit on the field.

Definition 3.2 (Effective wavenumber)**.** *Let $\widehat{\psi}(\mathbf{k})$ denote the spatial Fourier transform of $\psi(\cdot, t)$ at a fixed time. We say that ψ is* effectively band-limited *to radius $k_{\max}$ if*

$$\widehat{\psi}(\mathbf{k}) = 0 \quad \textit{whenever} \quad \|\mathbf{k}\| > k_{\max}. \tag{9}$$

We denote by k_{eff} an effective wavenumber scale associated to ψ, for example a suitable norm or average of $\mathbf{k}$ weighted by $\left|\widehat{\psi}\right|^2$.

Definition 3.3 (Curvature–wavenumber coupling)**.** *We assume that for confined WCT configurations the effective wavenumber is controlled by the curvature-rate average in the sense that there exists a constant $C_1 > 0$ with*

$$k_{\text{eff}} \leq C_1 \, \langle \sigma \rangle_w. \tag{10}$$

Combining the energy identity with the assumed coupling bounds the *effective* wavenumber, but does not by itself bound the maximum Fourier frequency or imply compact spectral support. A separate sharp bandwidth hypothesis is needed for finite-mode counting.

Lemma 3.4 (Effective wavenumber estimate; conditional sharp cutoff)**.** *Assume the rest-energy identity* (7)*, the coupling* (10)*, and* $E_{\mathrm{rest}}(\psi) \le E_{\max}$*. Then*

$$k_{\mathrm{eff}} \le C_1 \frac{E_{\max}}{\hbar c}. \tag{11}$$

If one additionally assumes *a uniform spectral-support bound* $\operatorname{supp} \widehat{\psi} \subseteq \{\|k\| \le k_{\max}\}$ *and the stronger coupling* $k_{\max} \le C_1 \langle \sigma \rangle_w$*, then*

$$k_{\max} \le C_1 \frac{E_{\max}}{\hbar c}. \tag{12}$$

The second conclusion is conditional on the sharp-support premise; it does not follow from the effective-wavenumber estimate alone.

Proof. The energy identity and bound give $\langle \sigma \rangle_w \le E_{\max}/(\hbar c)$. Substitution into the corresponding assumed coupling yields (11); using the *additional* sharp-cutoff coupling yields (12). □

We now translate the bandlimit into a bound on the number of independent spatial degrees of freedom ("lanes").

Lemma 3.5 (Curvature-bounded spatial channel capacity)**.** *Let* Ω *be a fixed periodic cubical cavity with volume* V*, and let the computation use only its Fourier modes satisfying* $\|\mathbf{k}\| \le k_{\max}$*. Assume* $k_{\max}$ *is at least the fundamental nonzero wavenumber of the cavity. Then there is a geometric constant* $C_2 > 0$ *such that the number* N_{lanes} *of these modes obeys*

$$N_{\mathrm{lanes}} \;\le\; C_2 \, V \, k_{\max}^3. \tag{13}$$

In particular, under the energy budget $E_{\mathrm{rest}}(\psi) \le E_{\max}$ *we have*

$$N_{\mathrm{lanes}} \;\le\; C_2 \, V \left(C_1 \frac{E_{\max}}{\hbar c} \right)^3. \tag{14}$$

Proof. For a periodic cubic cavity, allowed wavenumbers form a lattice of spacing proportional to $V^{-1/3}$. Counting the lattice points inside the spectral

ball gives, for $k_{\max}$ at least the fundamental wavenumber, an upper bound proportional to $Vk_{\max}^3$. The leading-volume approximation is

$$N_{\text{lanes}} \sim \frac{V}{(2\pi)^3}\,\frac{4\pi}{3}k_{\max}^3 = C_2\,V\,k_{\max}^3,$$

for an appropriate constant $C_2 > 0$. Substituting the bound (12) from Lemma 3.4 gives (14). □

Here N_{lanes} counts orthogonal spatial Fourier modes, not all metastable states in a nonlinear cavity. Its identification with independently addressable code lanes is an additional modeling hypothesis, not a general consequence of the mode count.

3.3 Temporal dimensional lock and spatiotemporal capacity

We now incorporate the temporal side of the dimensional lock. The key assumption is that curvature-locked code states cannot be switched arbitrarily fast.

Definition 3.6 (Dimensional lock in time)**.** *Let $\Delta t_{\min}$ denote the minimal dwell time required for a curvature-locked configuration to relax from one metastable code state to a distinct one while remaining within the WCT code manifold. We assume there exists a constant $C_3 > 0$ such that*

$$\Delta t_{\min} \;\geq\; \frac{C_3}{\omega_{\max}}, \tag{15}$$

where $\omega_{\max}$ is a characteristic maximal angular frequency of the band-limited field. For propagating modes we assume a dispersion relation of the form

$$\omega_{\max} \;\leq\; c\,k_{\max}, \tag{16}$$

with c the relevant propagation speed.

Lemma 3.7 (Per-lane temporal update bound)**.** *Under the assumptions of Lemma 3.5 and Definition 3.6, over any time window of duration $T > 0$ a single lane can undergo at most*

$$N_{\text{steps per lane}} \;\leq\; \frac{T}{\Delta t_{\min}} \;\leq\; \frac{T\,c\,k_{\max}}{C_3} \tag{17}$$

distinct metastable state transitions.

Proof. The first inequality is the definition of $N_{\text{steps per lane}}$. The second follows by combining (15) and (16):

$$\Delta t_{\min} \geq \frac{C_3}{\omega_{\max}} \geq \frac{C_3}{c\, k_{\max}} \quad \Longrightarrow \quad \frac{T}{\Delta t_{\min}} \leq \frac{T\, c\, k_{\max}}{C_3}.$$

□

Combining the spatial and temporal bounds yields a spatiotemporal capacity limit.

Theorem 3.8 (Curvature-bounded spatiotemporal channel capacity)**.** *Let $\Omega \subset \mathbb{R}^3$ be a bounded WCT cavity of volume V, confining a field ψ with rest-energy bounded by $E_{\max}$, and suppose the sharp spectral-cutoff, periodic-cavity, independent-lane, and minimum-dwell-time assumptions of Lemmas 3.4, 3.5, and 3.7 hold. Then, over any time window of duration $T > 0$, the total number N_{updates} of distinct metastable lane transitions obeys*

$$N_{\text{updates}} \;\leq\; C_4\, V\, T\, k_{\max}^4, \tag{18}$$

for some constant $C_4 > 0$, and hence

$$N_{\text{updates}} \;\leq\; K\, V\, T\, E_{\max}^4, \tag{19}$$

for a constant $K > 0$ depending only on $C_1, C_2, C_3, \hbar, c$.

Proof. By Lemma 3.5, the number of lanes satisfies

$$N_{\text{lanes}} \leq C_2\, V\, k_{\max}^3.$$

By Lemma 3.7, each lane can switch at most $Tck_{\max}/C_3$ times in the interval of duration T. Thus the total number of lane updates obeys

$$N_{\text{updates}} \;\leq\; N_{\text{lanes}} \cdot N_{\text{steps per lane}} \;\leq\; \Big(C_2\, V\, k_{\max}^3\Big)\Big(\frac{T\, c\, k_{\max}}{C_3}\Big) = C_4\, V\, T\, k_{\max}^4,$$

where $C_4 := C_2 c/C_3$. Substituting the energy-dependent bound $k_{\max} \leq C_1 E_{\max}/(\hbar c)$ from Lemma 3.4 yields

$$k_{\max}^4 \;\leq\; C_1^4\, \frac{E_{\max}^4}{\hbar^4 c^4},$$

and hence

$$N_{\text{updates}} \;\leq\; C_4\, V\, T\, k_{\max}^4 \;\leq\; C_4\, V\, T\, C_1^4 \frac{E_{\max}^4}{\hbar^4 c^4} \;=\; K\, V\, T\, E_{\max}^4$$

for $K := C_4 C_1^4/(\hbar^4 c^4)$. □

Remark 3.9 (Polynomial resource scaling)**.** *Conditional on the spectral-support, mode-counting, and minimum dwell-time assumptions, Theorem 3.8 shows that, for fixed physical resources* $(V, T, E_{\max})$*, the total number of resolvable metastable transitions in a WCT cavity is bounded by a polynomial in these parameters. In particular, if a family of WCT-based computing devices indexed by input size* n *is constrained so that*

$$V(n),\ T(n),\ E_{\max}(n)\ \leq\ \mathrm{poly}(n),$$

then the total number $N_{\mathrm{updates}}(n)$ *of elementary computational events across all lanes is also bounded by* $\mathrm{poly}(n)$*. Thus curvature confinement and dimensional lock enforce a physical bottleneck on the achievable computational complexity as a function of input size.*

4 Physics-Bounded Computation and Conditional NP Safety

In this section we combine the discrete WCC–Turing equivalence with the curvature/energy channel-capacity bound from the WCT model to obtain a conditional statement about the computational power of any AGI implemented on WCT hardware under the rails (L,P,V,C,S,R).

The next bound is a *conditional physical hypothesis* derived algebraically in the preceding section under the stated sharp-bandwidth, mode-counting, and dwell-time premises; it has not been established universally for arbitrary WCT PDE evolutions:

Theorem 4.1 (Conditional channel-capacity step bound)**.** *Let a WCT device occupy a cavity of volume* $V > 0$ *over a time interval* $[0, T]$ *with a rest-energy ceiling* $E_{\max} > 0$*. Assume the sharp spectral cutoff and the uniform mode-counting and minimum-dwell-time hypotheses of Section 3. Then the total number of distinct metastable lane updates (individual local state transitions)* N_{updates} *that can occur in* $[0, T]$ *obeys*

$$N_{\mathrm{updates}}\ \leq\ K\,V\,T\,E_{\max}^{4}, \tag{20}$$

for some constant $K > 0$ *depending only on the WCT parameters* $(\hbar, c, \xi, \dots)$ *and not on the particular device or algorithm.*

We now formalize what it means for a family of physical devices to be *polynomially bounded* in WCT resources.

Definition 4.2 (Polynomial-resource WCT device family)**.** *Let $(D_n)_{n\geq 1}$ be a family of WCT devices intended to process inputs of length n. For each n, let $V(n)$ denote the device volume, $T_{\mathrm{phys}}(n)$ the physical run time, and $E_{\max}(n)$ a rest-energy ceiling during the computation.*

We say that (D_n) is a polynomial-resource WCT device family *if there exist constants $C_V, C_T, C_E > 0$ and exponents $a, b, c \geq 0$ such that for all n,*

$$V(n) \;\leq\; C_V n^a, \qquad T_{\mathrm{phys}}(n) \;\leq\; C_T n^b, \qquad E_{\max}(n) \;\leq\; C_E n^c. \tag{21}$$

Combining Definition 4.2 with Theorem 4.1 yields a polynomial bound on the total number of usable primitive updates.

Proposition 4.3 (Physical step ceiling under WCT rails)**.** *Let $(D_n)_{n\geq 1}$ be a polynomial-resource WCT device family in the sense of Definition 4.2. Then there exist constants $C' > 0$ and $d \geq 0$ such that, for all n,*

$$N_{\mathrm{updates}}(n) \;\leq\; C' n^d. \tag{22}$$

In particular, any discrete WCC machine W faithfully implemented by such a device family (with every nonhalting global round incurring at least one counted elementary event) has worst-case runtime $T_W(n)$ bounded by a polynomial in n.

Proof. By Definition 4.2 we have

$$V(n) \leq C_V n^a, \quad T_{\mathrm{phys}}(n) \leq C_T n^b, \quad E_{\max}(n) \leq C_E n^c.$$

Plugging these into the channel-capacity bound (19) gives

$$\begin{aligned} N_{\mathrm{updates}}(n) &\leq K\, V(n)\, T_{\mathrm{phys}}(n)\, E_{\max}(n)^4 \\ &\leq K \cdot (C_V n^a) \cdot (C_T n^b) \cdot (C_E n^c)^4 \\ &= K C_V C_T C_E^4\; n^{\,a+b+4c}. \end{aligned}$$

Setting $C' := K C_V C_T C_E^4$ and $d := a + b + 4c$ yields (22). If every nonhalting global round incurs at least one counted elementary event, then $T_W(n) \leq N_{\mathrm{updates}}(n)+1$; therefore T_W is polynomially bounded. This counting premise must hold for the particular implementation and clocking convention. □

We now combine the physical step ceiling with the discrete model equivalences $P_{\mathrm{WCC}} = P_{\mathrm{Turing}}$ and $NP_{\mathrm{WCC}} = NP_{\mathrm{Turing}}$ from Sections 1–2 to obtain a conditional NP-safety statement.

Theorem 4.4 (Conditional NP-safety of WCT-bounded AGI)**.** *Assume:*

(i) The WCT rails (L,P,V,C,S,R) correctly describe the physically realizable dynamics of any WCT device, and the channel-capacity bound of Theorem 4.1 holds.

(ii) All physically realizable discrete computation on such devices is faithfully modeled by discrete WCC machines in the sense of Definition 1.1, with equivalences

$$P_{\mathrm{WCC}} = P_{\mathrm{Turing}}, \qquad NP_{\mathrm{WCC}} = NP_{\mathrm{Turing}}.$$

(iii) The classical complexity-theoretic separation $P_{\mathrm{Turing}} \neq NP_{\mathrm{Turing}}$ holds.

Let $(D_n)_{n\geq 1}$ be any polynomial-resource WCT device family (Definition 4.2) implementing an AGI-level computational system on inputs of length n. Then there exists an NP-complete language $L \subseteq \Sigma^*$ and a constant $k \geq 1$ such that, for infinitely many input lengths n, the system realized by (D_n) fails *to decide L on at least one input of length n using only resources obeying* (21)*. Equivalently, no such WCT-bounded AGI can decide all NP-complete languages within polynomial physical resources without violating at least one of assumptions (i)–(iii).*

Proof. Let L_{SAT} be any fixed NP-complete language (e.g. 3-SAT) over some finite alphabet Σ. By (iii), $P_{\mathrm{Turing}} \neq NP_{\mathrm{Turing}}$, so $L_{\mathrm{SAT}} \notin P_{\mathrm{Turing}}$. By (ii) and the NP-equivalence, $L_{\mathrm{SAT}} \notin P_{\mathrm{WCC}}$ as well. Unpacking the definition of P_{WCC}, this means that for every discrete WCC machine W deciding L_{SAT} and for every exponent $k \geq 1$, there exist infinitely many input lengths n such that $T_W(n) > n^k$.

Now let (D_n) be a polynomial-resource WCT device family implementing an AGI-level system that, by assumption (ii), corresponds to some WCC machine W_{AGI} on inputs of length n. By Proposition 4.3, there exist constants C', d such that $T_{W_{\mathrm{AGI}}}(n) \leq C' n^d$ for all n, because each global WCC update costs at most a constant number of lane updates and the total number of lane updates is polynomially bounded.

Suppose for contradiction that this AGI decides L_{SAT} on all inputs of length n while respecting the polynomial resource bounds (21). Then W_{AGI} decides L_{SAT} with runtime $T_{W_{\mathrm{AGI}}}(n) \leq C' n^d$, so $L_{\mathrm{SAT}} \in P_{\mathrm{WCC}}$, and hence $L_{\mathrm{SAT}} \in P_{\mathrm{Turing}}$ by (ii), contradicting the choice of L_{SAT} and assumption (iii).

Therefore, for any polynomial-resource WCT device family (D_n), there must exist some NP-complete language L (in fact any NP-complete language) and some $k \geq 1$ such that, for infinitely many lengths n, the system implemented by (D_n) cannot decide L on at least one input of length n without exceeding the polynomial resource bounds in (21) or violating at least one of (i)–(iii). This is precisely the claimed conditional NP-safety statement. □

Remark 4.5 (Interpretation for AGI safety)**.** *Theorem 4.4 can be read as follows. If* (a) *the WCT rails correctly bound all physically realizable computation and* (b) *the classical separation $P \neq NP$ holds, then no AGI implemented on WCT hardware with volume, time, and energy scaling at most polynomially in the input size can solve NP-complete problems on all inputs within those resource bounds. In particular, such an AGI cannot "cheat" cryptography or other NP-hard tasks by exploiting super-Turing parallelism or infinitely fine analog degrees of freedom: any attempt to do so would require either super-polynomial physical resources or a violation of the WCT rails.*

5 Uniform Encodings, Decision Equivalence, and the Physical Bridge

The preceding sections establish an equivalence of *language classes* between uniform discrete local-update WCC machines and standard Turing machines. They do not establish a pointwise bijection between every abstract Turing configuration and every continuous wavefield. In particular, a general polynomial-time verifier need not be in NC^1: the WCC machine simulates the full verifier computation using polynomially many local rounds.

5.1 Uniform input and certificate representations

For each fixed classical deterministic machine M, the construction of Section 1 supplies a polynomial-time injective tape-to-lattice encoding $\mathrm{enc}_M(x)$ and a fixed-radius local transition rule implementing one classical Turing

step in a constant number of WCC rounds. The readout of a halting state is polynomial-time uniform. For a classical verifier $V_{\mathrm{TM}}(x, w)$, use the same uniform construction on an efficiently delimited encoding of the pair (x, w).

Lemma 5.1 (Preservation of verifier outcomes)**.** *For every fixed polynomial-time Turing verifier V_{TM}, there exists a uniform discrete WCC verifier V_{WCC} satisfying*

$$V_{\mathrm{TM}}(x, w) = 1 \quad \Longleftrightarrow \quad V_{\mathrm{WCC}}(\mathrm{enc}(x, w)) = 1$$

for every input and candidate witness, with polynomially bounded encoding, time, lattice size, and decoding overhead.

Proof. Apply the two-phase radius-one Turing simulation to the fixed verifier V_{TM}. Its transition rule does not depend on x or w, and the input pair is encoded uniformly. The simulation invariant preserves the simulated tape, head, and halting state, so acceptance agrees. Polynomial runtime and tape usage of the verifier give polynomial overhead for the WCC representation. □

5.2 Decision-class equivalence

Theorem 5.2 (Uniform discrete complexity equivalence)**.** *For the uniform finite-state model of Definition 1.1,*

$$P^{\mathrm{disc}}_{\mathrm{WCC}} = P_{\mathrm{TM}}, \qquad NP^{\mathrm{disc}}_{\mathrm{WCC}} = NP_{\mathrm{TM}}.$$

The equalities identify the sets of recognized languages *and are not a claim that arbitrary encodings are bijective or that physically constrained WCT evolution realizes every such machine.*

Proof. The deterministic equality is Corollary 1.18; the nondeterministic verifier equality is Theorem 2.5. Both rely on polynomial-time uniform input encoding and decoding. □

5.3 Conditional lifting, without conflating models

Proposition 5.3 (Classical implication under an actual common-model separation)**.** *If a language L is proved to satisfy $L \in NP^{\mathrm{disc}}_{\mathrm{WCC}} \setminus P^{\mathrm{disc}}_{\mathrm{WCC}}$, then $P \neq NP$.*

Proof. Substitute the discrete class equalities from Theorem 5.2. □

A lower bound for only *physical* curvature-bounded realizations does not satisfy the premise of this proposition. Deriving classical $P \neq NP$ from a physical separation would require an independently proved equality of the *same* physical and discrete decision and verifier classes, with polynomial preservation of all relevant costs. No such physical-to-discrete resource equivalence or classical separation is established here.

Formal verification status (November 2026). For the one-dimensional, radius-one, finite-alphabet, quiescent discrete model, both simulation directions and the class equalities of Theorem 5.2 are kernel-checked in the accompanying Lean project (`PVNP.Discrete.P_WCC_eq_P_TM`, `PVNP.Discrete.NP_WCC_eq_NP_TM`). The Lean Turing-to-WCC simulation runs in real time (overhead factor one); the WCC-to-Turing simulation is an explicit single-tape machine using alternating shifted sweeps, with at most $1 + (T(n) + 1)(n + 2T(n) + 5)$ steps. Radius-r and higher-dimensional lattices and non-local polynomial-time encoders are covered by the analytic argument above but are not formalized. The formal transfer theorem `PVNP.Discrete.p_vs_np_discrete_transfer` shows that the discrete question $P^{\mathrm{disc}}_{\mathrm{WCC}} \stackrel{?}{=} NP^{\mathrm{disc}}_{\mathrm{WCC}}$ is exactly the classical one; it is not resolved here.

6 Conclusion

The principal rigorous result is the polynomial-time equivalence of *uniform discrete* WCC machines and classical Turing machines. Fixed finite-state local rules, polynomial-size constructible lattices, polynomial-time input encoders and output decoders, and polynomially many global update rounds yield

$$P^{\mathrm{disc}}_{\mathrm{WCC}} = P, \qquad NP^{\mathrm{disc}}_{\mathrm{WCC}} = NP.$$

The uniformity requirements exclude arbitrary noncomputable preprocessing in the encoding map. Exact wave-to-symbol correspondence requires quantization compatibility; the converse construction of an abstract finite-state representative evolution does not establish realization by a chosen nonlinear WCT PDE.

The physical curvature/energy channel-counting estimates are presented conditional on their additional spectral-support and dynamical assumptions. Neither those estimates nor the discrete Turing simulations prove $P^{\mathrm{phys}}_{\mathrm{WCC}} = P$

or $NP^{\text{phys}}_{\text{WCC}} = NP$, and no unconditional $P \neq NP$ or universal SAT lower bound is claimed in this manuscript.

The remaining research problem is to construct and validate a precise physical realization map with uniform polynomial resource overhead, then separately study any model-relative physical resource lower bounds.

References

1. S. A. Cook. "The Complexity of Theorem-Proving Procedures." *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 1971.

2. L. A. Levin. "Universal Sequential Search Problems." *Problems of Information Transmission*, 9(3):265–266, 1973.

3. T. Baker, J. Gill, R. Solovay. "Relativizations of the P=?NP Question." *SIAM Journal on Computing*, 4(4):431–442, 1975.

4. A. A. Razborov, S. Rudich. "Natural Proofs." *Journal of Computer and System Sciences*, 55(1):24–35, 1997.

5. S. Aaronson, A. Wigderson. "Algebrization: A New Barrier in Complexity Theory." *ACM Transactions on Computation Theory*, 1(1):2:1–2:54, 2009.

6. A. Lubotzky, R. Phillips, P. Sarnak. "Ramanujan Graphs." *Combinatorica*, 8(3):261–277, 1988.

7. S. Arora, B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

8. R. J. Reyes. *Structure and Derivation of Physical Constants through Wave Confinement*. Zenodo, 2025. `https://doi.org/10.5281/zenodo.15341540`.

9. R. J. Reyes. *The Geometry of Resonance: Wave Confinement Theory and the Emergence of Mass, Force, and Spacetime*. Zenodo, 2025. `https://doi.org/10.5281/zenodo.15356814`.